01 - Geth 시스템 개요 및 아키텍처

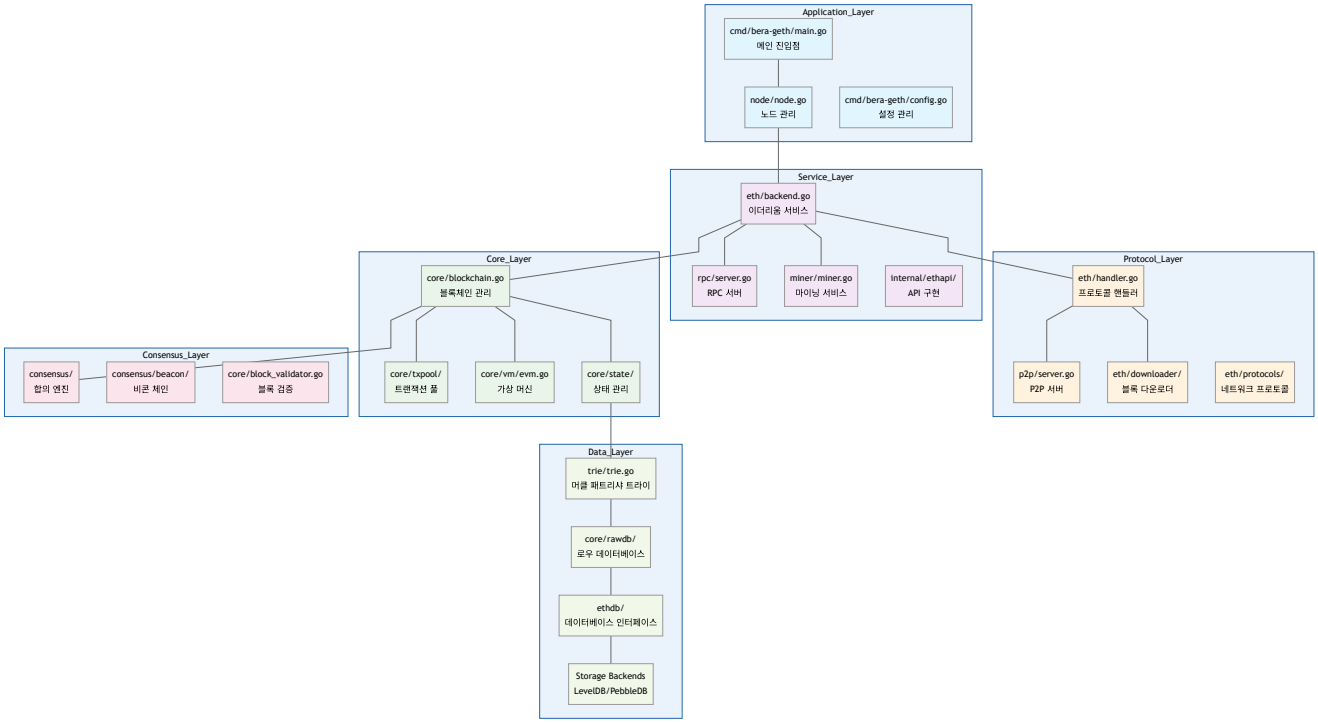
개요

Go-Ethereum (Geth) 구현체는 이더리움 프로토콜의 완전한 구현체로, 복잡한 분산 시스템 아키텍처를 가지고 있습니다. 본 문서는 Geth의 전체 시스템 아키텍처를 설명하고, 각 핵심 컴포넌트들이 어떻게 상호작용하는지 분석합니다.

Geth는 계층화된 아키텍처를 통해 블록체인의 핵심 기능들을 모듈화하여 구현하고 있으며, 각 계층은 명확한 책임과 인터페이스를 가지고 있습니다.

전체 시스템 아키텍처

시스템 계층 구조


Application_Layer
cmd/bera-geth/main.go
메인 진입점
node/node.go
노드 관리
cmd/bera-geth/config.go
설정 관리
Service_Layer
eth/backend.go
이더리움 서비스
rpc/server.go
RPC 서버
miner/miner.go
마이닝 서비스
internal/ethapi/
API 구현
Core_Layer
core/blockchain.go
블록체인 관리
core/txpool/
트랜잭션 풀
core/vm/evm.go
가상 머신
core/state/
상태 관리
Protocol_Layer
eth/handler.go
프로토콜 핸들러
p2p/server.go
P2P 서버
eth/downloader/
블록 다운로더
eth/protocols/
네트워크 프로토콜
Consensus_Layer
consensus/
합의 엔진
consensus/beacon/
비콘 체인
core/block_validator.go
블록 검증
Data_Layer
trie/trie.go
머클 패트리샤 트라이
core/rawdb/
로우 데이터베이스
ethdb/
데이터베이스 인터페이스
Storage Backends
LevelDB/PebbleDB

핵심 구조체 및 컴포넌트 매핑

1. 노드 관리 (`node/` 디렉터리)

`node.Node` 구조체

위치: `node/node.go`

```
type Node struct {
eventmux *event.TypeMux // 이벤트 멀티플렉서
config *Config // 노드 설정
accman *accounts.Manager // 계정 관리자
server *p2p.Server // P2P 서버
rpcAPIs []rpc.API // RPC API 목록
http *httpServer // HTTP 서버
ws *httpServer // WebSocket 서버
// ... 기타 필드들
}
```

주요 메서드:

- `New(conf *Config) (*Node, error)` - 노드 생성
- `Start() error` - 노드 시작
- `Stop() error` - 노드 중단
- `RegisterAPIs(apis []rpc.API)` - API 등록
- `OpenDatabase(name string) (ethdb.Database, error)` - 데이터베이스 열기

2. 이더리움 서비스 (`eth/` 디렉터리)

`eth.Ethereum` 구조체

위치: `eth/backend.go`

```
type Ethereum struct {
config *ethconfig.Config // 이더리움 설정
txPool *txpool.TxPool // 트랜잭션 풀
blockchain *core.BlockChain // 블록체인
handler *handler // 프로토콜 핸들러
engine consensus.Engine // 합의 엔진
accountManager *accounts.Manager // 계정 관리자
miner *miner.Miner // 마이너
// ... 기타 필드들
}
```

주요 메서드:

- `New(stack *node.Node, config *ethconfig.Config) (*Ethereum, error)` - 이더리움 서비스 생성

- `Start() error` - 서비스 시작
- `Stop() error` - 서비스 중단
- `Protocols() []p2p.Protocol` - 지원 프로토콜 반환

시스템 초기화 과정

1. 메인 함수에서 시작 (`cmd/bera-geth/main.go`)

```
func main() {
if err := app.Run(os.Args); err != nil {
fmt.Fprintln(os.Stderr, err)
os.Exit(1)
}
}
```

초기화 흐름:

1. CLI 애플리케이션 실행
2. `berageth()` 함수 호출
3. `prepare()` - 네트워크 설정 및 캐시 준비
4. `makeFullNode()` - 전체 노드 생성
5. `startNode()` - 노드 시작

2. 노드 생성 과정 (`cmd/bera-geth/config.go`)

```
func makeFullNode(ctx *cli.Context) *node.Node {
```

```
stack, cfg := makeConfigNode(ctx)
// 메트릭 설정
utils.SetupMetrics(&cfg.Metrics)
// 이더리움 서비스 등록
backend, eth := utils.RegisterEthService(stack, &cfg.Eth)
// 필터 시스템 등록
filterSystem := utils.RegisterFilterAPI(stack, backend, &cfg.Eth)
// GraphQL 서비스 등록 (선택적)
if ctx.IsSet(utils.GraphQLEnabledFlag.Name) {
utils.RegisterGraphQLService(stack, backend, filterSystem, &cfg.Node)
}
return stack
}
```

3. 이더리움 서비스 등록 (`cmd/utils/flags.go`)

```
func RegisterEthService(stack *node.Node, cfg *ethconfig.Config) (*eth.EthAPIBa
backend, err := eth.New(stack, cfg)
if err != nil {
Fatalf("Failed to register the Ethereum service: %v", err)
}
stack.RegisterAPIs(tracers.APIs(backend.APIBackend))
return backend.APIBackend, backend
```

```
}
```

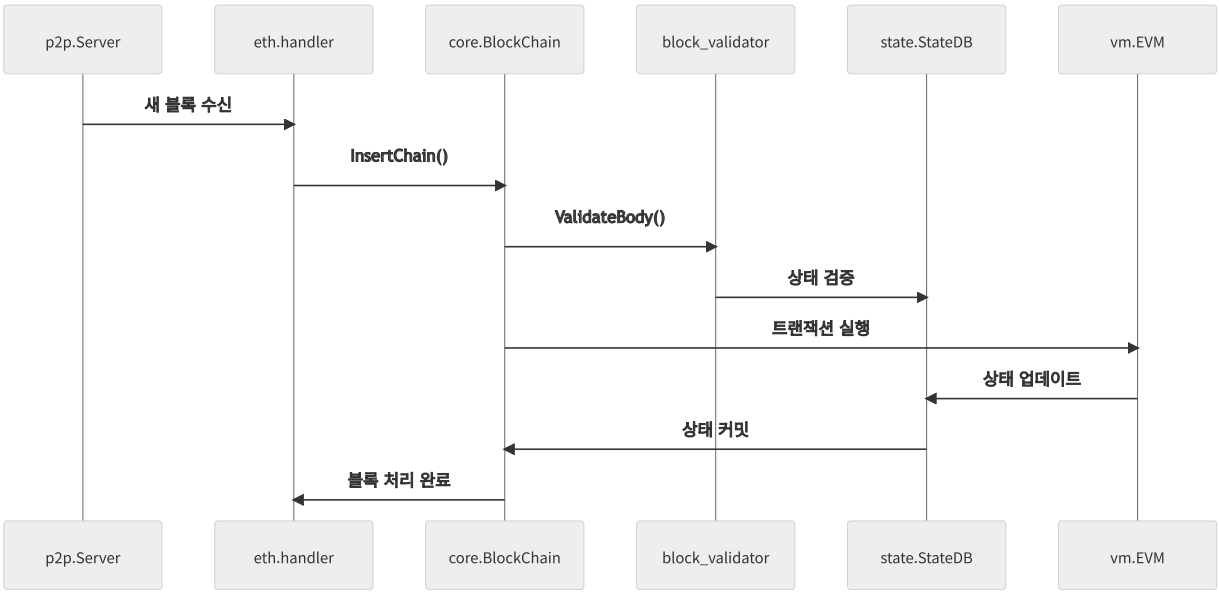
4. 이더리움 서비스 생성 (eth/backend.go)

```
func  New(stack *node.Node, config *ethconfig.Config) (*Ethereum, error) {
// 1. 데이터베이스 열기
chainDb, err := stack.OpenDatabaseWithOptions("chaindata", dbOptions)
// 2. 체인 설정 로드
chainConfig, _, err := core.LoadChainConfig(chainDb, config.Genesis)
// 3. 합의 엔진 생성
engine, err := ethconfig.CreateConsensusEngine(chainConfig, chainDb)
// 4. 블록체인 생성
blockchain, err := core.NewBlockChain(chainDb, cacheConfig, config.Genesis,
&overrides, engine, vmConfig,
config.HistoryMode, config.StateScheme)
// 5. 트랜잭션 풀 생성
txPool, err := txpool.New(config.TxPool, chainConfig, blockchain)
// 6. 이더리움 객체 조립
eth := &Ethereum{
config: config,
chainDb: chainDb,
blockchain: blockchain,
txPool: txPool,
```

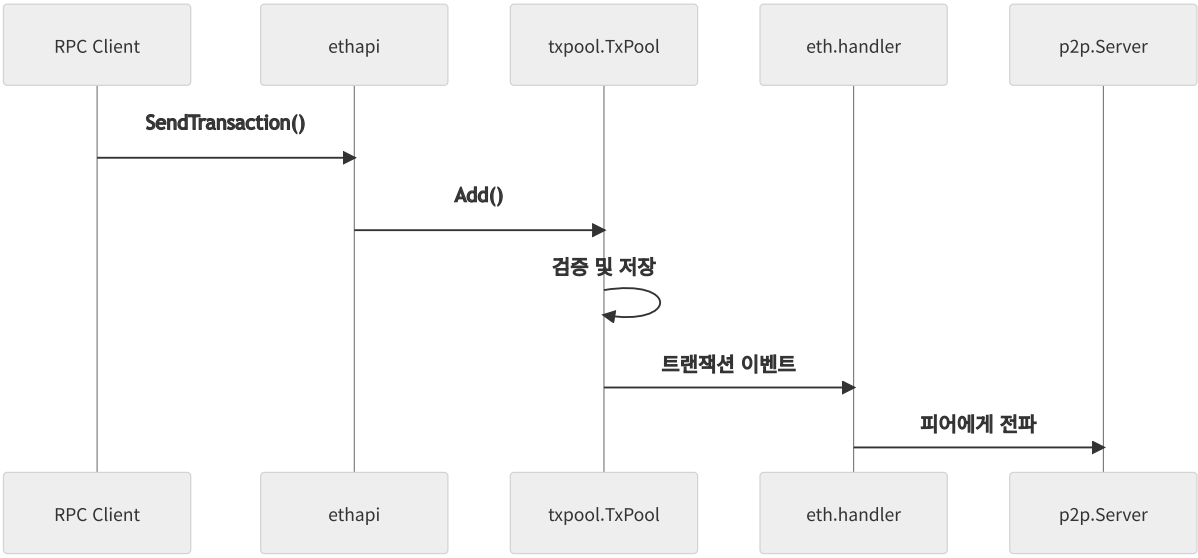
```
engine: engine,

// ... 기타 필드 초기화

}

return  eth, nil

}
```

컴포넌트 간 상호작용

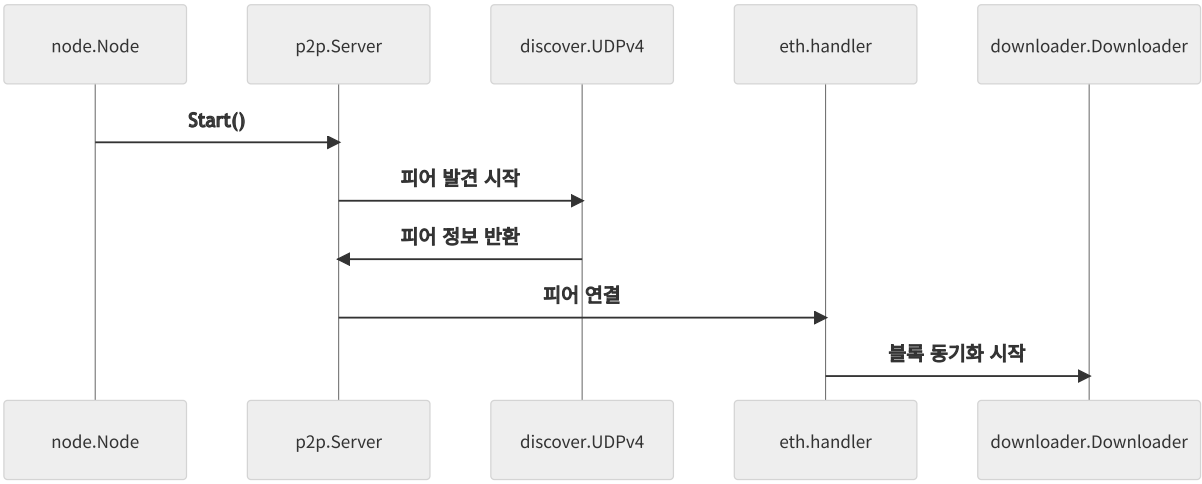
1. 블록 처리 흐름



2. 트랜잭션 처리 흐름



3. P2P 네트워킹 흐름



디렉터리별 핵심 컴포넌트 매핑

애플리케이션 계층

- **`cmd/bera-geth/`** : 메인 애플리케이션 진입점
- `main.go` → `main()` 함수
- `config.go` → `makeFullNode()`, `makeConfigNode()`
- **`node/`** : 노드 생명주기 관리
- `node.go` → `node.Node` 구조체
- `config.go` → 노드 설정 관리

서비스 계층

- **`eth/`** : 이더리움 프로토콜 서비스
- `backend.go` → `eth.Ethereum` 구조체
- `api_backend.go` → API 백엔드 구현
- **`rpc/`** : RPC 서버 및 클라이언트
- `server.go` → `rpc.Server` 구조체
- `client.go` → `rpc.Client` 구조체

프로토콜 계층

- **`eth/handler.go`** : 이더리움 프로토콜 핸들러
- **`p2p/`** : P2P 네트워킹
- `server.go` → `p2p.Server` 구조체
- `discover/` → 피어 발견 메커니즘

코어 계층

- **`core/`** : 블록체인 핵심 로직
- `blockchain.go` → `core.BlockChain` 구조체
- `state/` → 상태 관리 시스템
- `vm/` → 이더리움 가상 머신
- `txpool/` → 트랜잭션 풀 관리

합의 계층

- **`consensus/`** : 합의 엔진 구현
- `consensus.go` → `consensus.Engine` 인터페이스
- `beacon/` → 비콘 체인 합의

데이터 계층

- **`trie/`** : 머클 패트리샤 트라이
- **`ethdb/`** : 데이터베이스 인터페이스
- **`core/rawdb/`** : 로우 데이터베이스 접근

시스템 시작 순서

1. 초기화 단계

1. **CLI 파싱**: 명령행 인수 처리
2. **설정 로드**: 네트워크 및 노드 설정
3. **노드 생성**: `node.Node` 인스턴스 생성
4. **데이터베이스 열기**: 체인데이터 데이터베이스 초기화

2. 서비스 등록 단계

1. **이더리움 서비스**: `eth.Ethereum` 생성 및 등록
2. **RPC 서비스**: API 엔드포인트 등록
3. **필터 시스템**: 로그 필터링 서비스 등록
4. **추가 서비스**: GraphQL, 통계 서비스 등

3. 시작 단계

1. **노드 시작**: `node.Start()` 호출
2. **P2P 서버 시작**: 네트워크 연결 수락
3. **프로토콜 핸들러 시작**: 이더리움 프로토콜 처리
4. **동기화 시작**: 블록체인 동기화 프로세스

4. 운영 단계

1. **피어 연결**: 다른 노드와 연결 설정
2. **블록 동기화**: 최신 블록 다운로드
3. **트랜잭션 처리**: 트랜잭션 풀 관리
4. **API 서비스**: 외부 클라이언트 요청 처리

주요 인터페이스 및 추상화

1. 서비스 인터페이스

```
type Lifecycle interface {
Start() error
```

```
Stop() error
}
```

2. 합의 엔진 인터페이스

```
type Engine interface {
VerifyHeader(chain ChainHeaderReader, header *types.Header) error
Prepare(chain ChainHeaderReader, header *types.Header) error
Finalize(chain ChainHeaderReader, header *types.Header, state *state.StateDB,
txs []*types.Transaction, uncles []*types.Header,
withdrawals []*types.Withdrawal) error
}
```

3. 데이터베이스 인터페이스

```
type Database interface {
Get(key []byte) ([]byte, error)
Put(key []byte, value []byte) error
Delete(key []byte) error
NewBatch() Batch
}
```

성능 및 확장성 고려사항

1. 모듈화 설계

- 각 컴포넌트는 독립적인 인터페이스를 통해 상호작용
- 의존성 주입을 통한 느슨한 결합
- 플러그인 아키텍처로 확장 가능

2. 비동기 처리

- 이벤트 기반 아키텍처
- 고루틴을 활용한 동시성 처리
- 채널을 통한 컴포넌트 간 통신

3. 캐싱 전략

- 다층 캐시 구조
- LRU 캐시 알고리즘 활용
- 메모리 사용량 최적화

디버깅 및 모니터링

1. 로깅 시스템

- 구조화된 로깅 (structured logging)
- 로그 레벨별 필터링
- 컴포넌트별 로그 네임스페이스

2. 메트릭 수집

- Prometheus 메트릭 지원

- 성능 지표 실시간 모니터링
- 시스템 리소스 사용량 추적

3. 프로파일링

- CPU 및 메모리 프로파일링
- 고루틴 누수 감지
- 성능 병목 지점 분석

이 아키텍처 개요를 통해 Geth의 전체적인 구조와 각 컴포넌트의 역할을 이해할 수 있습니다. 다음 문서들에서는 각 계층의 세부 구현을 더 자세히 살펴보겠습니다.